

CS F407 – Agents Lab: Constructing a Goal-Based Agent with an LLM

This report presents the solution to the laboratory exercise [agents_lab.pdf](#).

File	Contents
goal_based_agent.py	Environment, goal-based agent, BFS planner, validation, and scaling experiment
results.txt	Full output of <code>python goal_based_agent.py</code> ; every number in this report is taken from it
README.md / REPORT.pdf	This report, in Markdown and PDF form

Requirements: Python 3 (standard library only). To reproduce every result:

```
python goal_based_agent.py
```

Task 1 – Understanding the problem

1. What is the environment? A warehouse floor modelled as a 21×7 grid. Shelving units (#) cannot be crossed, and free cells (.) can be traversed. The vehicle starts at $S = (1, 1)$ and must reach the dispatch area $G = (1, 19)$. The environment is fully observable, deterministic, static, discrete, and single-agent.

2. What is the goal of the agent? To move the vehicle from S to G without entering any obstacle cell, preferably by a shortest route.

3. What actions are available? Up, Down, Left, and Right. Each moves the vehicle by one grid square; a move into a shelf is not permitted.

4. What information must the agent maintain?

- its current position (state);
- the goal position;
- a model of the warehouse, i.e. which cells are free;
- the plan computed so far, i.e. the remaining actions.

During planning it also needs the search frontier and the set of visited cells, so that it does not revisit positions.

5. Why is this a goal-based agent rather than a simple reflex agent? A simple reflex agent maps the current percept directly to an action through condition–action rules, such as "if the cell to the right is free, move right". Such rules have no notion of where the agent is trying to go. On this map a rule such as "move towards G" leads into the shelf at (1, 6), and the correct route requires a temporary move *down*, away from the goal's row. A goal-based agent holds an explicit goal and reasons about the future

consequences of action sequences through search. It can therefore select actions that do not look locally best but lead to the goal.

Think About It – a warehouse twice as large. The experiment enlarged the map by factors of 2, 4, and 8 (each cell replicated $k \times k$ times) and compared BFS with A* using the Manhattan heuristic:

Map	Free cells	BFS states expanded	A* states expanded	Path length	BFS time (ms)
1×	66	59	23	20	0.07
2×	264	237	80	40	0.31
4×	1,056	935	296	80	1.16
8×	4,224	3,723	1,136	160	5.14

The times depend on the machine and are indicative only.

BFS remains correct and optimal at every size, and at a scale of 2× it is still entirely adequate. However, its work grows with the *area* of the map (about 4 times more expansions per doubling), whereas the path length grows only linearly. A* with an admissible heuristic returned paths of the same length while expanding roughly a third as many states. For larger warehouses an informed search such as A* is therefore more appropriate.

Further difficulties at scale:

- memory for the frontier and visited set;
- replanning cost when shelves or other vehicles move (a dynamic environment);
- coordination between multiple vehicles;
- partial observability if the map is not fully known in advance.

Task 2 – Design of the agent

Component	Design
Environment	<code>WarehouseEnvironment</code> : holds the true grid and the vehicle's true position, supplies percepts, and executes actions, rejecting and counting any move into a shelf
Current state	The vehicle position <code>(row, col)</code> , obtained from the percept
Goal	The G cell, stored explicitly in the agent
Actions	Up, Down, Left, Right (<code>MOVES</code>)
Decision-making component	<code>GoalBasedAgent.decide()</code> : if no plan exists, it runs breadth-first search over its internal model to obtain an action sequence from the current state to the goal; it then returns the next action of the plan

Block diagram


```
#####
#S**.*#####G#
#.#**.*#####.#
#...#.#.....#
#.######.###.#
#.....#.....#
#####
```

Actions: Right $\times 3$, Down, Right $\times 3$, Up, Right $\times 12$.

Run 2 – validation. An independent check confirmed that the path starts at S, ends at G, visits only free cells, and moves exactly one cell per step. The Manhattan distance from S to G is 18. The path length of 20 is consistent with the unavoidable two-move detour around the shelf at (1, 6), and because BFS is optimal for unit step costs, 20 is the minimum.

Run 3 – no path exists. The two cells adjacent to G were turned into shelves. The agent explored all 63 reachable cells and reported "*no collision-free path exists*" instead of moving or looping indefinitely.

Answers

1. Did the LLM generate a working program on the first attempt? Yes. The program ran without errors and produced a valid, optimal path on the first execution. It was then extended with an explicit independent path check, an unreachable-goal test, and the scaling experiment. These additions were made so that correctness was *demonstrated* rather than assumed from a single plausible output.

2. How could the prompt be improved?

- State the move model and cost explicitly (four directions, cost 1 per move), so that the algorithm choice can be justified.
- Require an agent structure (perceive–decide–act), not merely a path-finding function.
- Request test cases: an unreachable goal, a trivial one-step map, and an independent validity check of the returned path.
- Specify the output format (path as coordinates and as actions, path length, number of states expanded).
- Ask the LLM to state any assumptions it makes, for example about diagonal moves.

3. Which search algorithm did the LLM choose? Breadth-first search (BFS).

4. Why did the LLM select this algorithm? Every move has the same cost, so BFS is **complete** (it always finds a path if one exists on a finite grid) and **optimal** (the first path to reach G has the fewest moves). It is also simple to implement and needs no heuristic. For a map of 66 free cells its cost is negligible. The scaling experiment shows that this choice should be revisited for much larger warehouses, where A* with an admissible heuristic gives the same optimal paths with considerably fewer expansions.